

Documentation Mind-Map - Reusable Prompt

Purpose: Drop this into a Claude Code session pointed at any documentation corpus (a /docs folder, an exported Confluence space, an ADR archive, an onboarding wiki, a regulatory document set). It dispatches a team of subagents to read everything in parallel, then produces a navigable knowledge base - an interactive HTML mind-map AND a markdown index - with every concept mapped back to the source document it came from.

Where it came from: Companion to the Repo Architecture Mapping Prompt cheatsheet (which targets source code). Same parallel-subagent pattern, but tuned for prose: the unit of exploration is a document or topic, not a module or file.

Why parallel subagents: Reading a 200-page documentation set sequentially exhausts context and produces a flat narrative. Parallel subagents each get a focused topic partition (architecture, operations, security, onboarding, etc.) and return concentrated structured findings. The orchestrator merges them into a single mind-map with cross-references. Better artifacts, much faster wall-clock.

Why TWO outputs: The HTML mind-map is for navigation - a manager scanning the corpus, an engineer onboarding to a new system. The markdown INDEX + per-topic notes are for diff, code review, and grep - git-friendly artifacts a team can maintain alongside the docs themselves. The HTML is the "read it once" artifact; the markdown is the "live with it" artifact.

Works with /goal: the prompt is structured around a single goal statement; set it as the session goal, paste the body, and Claude Code's goal-tracking will follow progress as the orchestrator dispatches and the subagents return.

How to use

Step 1 - Set the goal (optional but recommended)

Inside a Claude Code session at the root of the documentation tree:

```
/goal Build a knowledge base of this documentation corpus using parallel subagents. Produce an interactive HTML mind-map and a markdown INDEX + per-topic notes. Every claim cites a source document with file:line.
```

Step 2 - Paste the prompt

Copy the entire block below into the Claude Code session. The agent will read GOAL + APPROACH and start dispatching.

The prompt

GOAL: Build a navigable knowledge base from this documentation corpus.
Parallel subagent exploration. Dual output: interactive HTML mind-map

+ markdown INDEX with per-topic notes. Every claim cites a source document and line range. Validation pass at the end.

APPROACH (orchestrator -> subagents -> orchestrator):

1. ORCHESTRATOR FIRST PASS (~2 min max).

Walk the top-level directory listing. Read any README.md, INDEX.md, TOC.md, or top-level overview. Identify document types (markdown, pdf, docx, html, txt), rough document count, and the corpus's apparent organizing principle (by-topic, by-team, by-system, by-timeline, by-audience). Note the size: count files, count total pages or line-count if available.

2. DECOMPOSE INTO 4-7 TOPIC PARTITIONS.

Pick what fits THIS corpus. Default partitions (adjust based on what you actually see):

- Architecture & design (system overviews, diagrams, ADRs, decision records)
- Operations (runbooks, on-call, incident response, deployment)
- Security & compliance (threat models, policies, audit notes, regulatory mappings)
- Onboarding & how-to (getting-started guides, tutorials, dev setup, FAQs)
- APIs & contracts (API references, OpenAPI specs, integration guides, SLAs)
- Domain knowledge (business glossaries, domain models, product requirements, user research)
- Process & governance (engineering standards, code review protocols, release process)

If the corpus is small (<20 documents) or single-domain (a tutorial set, a single product's API ref), use 2-3 partitions. If the corpus is huge (>500 documents) or multi-system, partition by sub-system first, then by topic within each sub-system.

3. DISPATCH ONE SUBAGENT PER PARTITION via the Task tool.

Each subagent gets:

- Partition name + one-sentence purpose
- 1-3 starting directories or document patterns (globs)
- Output contract: structured markdown with three sections -
 - (a) Topics covered: a table with "Topic | One-sentence summary | Source doc (path:line range) | Confidence (high/med/low)"
 - (b) Cross-references: topics that touch other partitions (e.g., "security policy X is enforced via runbook Y")
 - (c) Gaps & open questions: missing docs, contradictions between docs, stale dates, unresolved TODOs
- Quality bar: every topic has a source citation. No claims without citations. Don't invent topics that aren't in the docs. If a topic is implied but not documented, flag it as a Gap. For PDFs and other binary formats, cite page numbers.

4. SYNTHESIZE WHEN ALL RETURN.

- Build a consolidated topic graph: nodes are topics, edges are cross-references between partitions.
- Deduplicate: if two subagents covered the same topic, merge into one node and keep both citations.
- Resolve contradictions: when two subagents report different things about the same topic, surface as an Open Question in the final output rather than picking a winner.
- Identify the 5-10 highest-traffic topics (the ones with the most cross-references) - these are the corpus's "anchor concepts."

5. GENERATE BOTH ARTIFACTS.

(a) Interactive HTML at `./knowledge-base/mindmap.html`

Required structure:

- Header: corpus name (from git remote, top-level README, or the directory name), document count, generation timestamp.
- Anchor concepts section at the top: the 5-10 highest-traffic topics, each a clickable card linking to its full notes file.
- Mind-map tree: one branch per partition, color-coded; each topic is a leaf with a one-line summary and clickable links to the source doc(s) AND to the per-topic markdown note.
- Cross-reference graph at the bottom: a flat list of partition-spanning links ("topic X in [partition A] relates to topic Y in [partition B] via [source citation]"). Use a simple table - no graph library required.
- Footer: list of documents NOT covered (deliberately or missed), Open Questions, generation metadata (subagent count, total docs surveyed, link resolve rate).
- Simple HTML + CSS, no external dependencies, no JS frameworks. Plain expandable `<details>` tags for tree nodes are fine. Must open standalone in a browser.

(b) Markdown INDEX at `./knowledge-base/INDEX.md`

+ per-topic notes at `./knowledge-base/notes/<topic-slug>.md`

INDEX.md structure:

- One section per partition with a table:
"Topic | One-line summary | Notes file | Sources"
- Anchor concepts list at the top with links to the notes files.
- Cross-reference list at the bottom.
- Open Questions section.

Per-topic note file structure (one file per topic):

- Frontmatter: topic name, partition, confidence, last-updated date (today's date), source list with paths and line/page ranges.
- Body: 100-300 words capturing what the docs say. Direct quotes for definitions and policies. Paraphrase for narrative.
- "See also" section linking to related topic notes via relative paths.
- "Open questions" section if any from the subagent report.

6. VALIDATE ALL LINKS AND CITATIONS.

Walk the generated HTML and INDEX.md, extract every link and citation, confirm:

- file:// links point to real files on disk
- relative markdown links resolve to real notes files
- source citations match the path:line claimed (sample-check 10% by opening the cited doc and confirming the topic is actually there)

Report total counts and resolve rates.

CONSTRAINTS:

- Read-only on the corpus. Never modify any source document.
- The output goes ONLY to `./knowledge-base/` (a new folder). Don't scatter notes alongside the source docs.
- If a doc is encrypted, paywalled, or unreadable (corrupt PDF), skip it and list it in the "uncharted" footer.
- If you find sensitive data (credentials, PII) in the docs, redact it in your notes and flag it as a high-severity Open Question - don't propagate secrets into the knowledge base.

- Use standard file:// links: absolute paths, no trailing punctuation, URL-encode spaces. Note in the generated HTML footer that Chrome (and Edge) block file:// -> file:// nav by default; launch with --allow-file-access-from-files and an isolated --user-data-dir profile to enable clickthrough.

OUTPUT:

- ./knowledge-base/mindmap.html (single-file, opens standalone)
- ./knowledge-base/INDEX.md
- ./knowledge-base/notes/*.md (one file per topic)
- A summary message in the chat: subagents dispatched, total docs surveyed, total topics identified, anchor-concept list, link resolve rate, top 5 Open Questions worth following up.

What you get back

Three artifacts in a single new folder:

- knowledge-base/mindmap.html - open in a browser. Chrome (and Edge) block file:// -> file:// navigation by default - launch with --allow-file-access-from-files --user-data-dir=/tmp/chrome-mindmap to enable clickthrough into source docs (the /tmp/ profile isolates the security-relaxed session from your normal browsing). The "read it once" view: scan the anchor concepts, click into the partitions you care about, follow citations back to the source docs.
- knowledge-base/INDEX.md - the "live with it" view: grep-friendly, diffable, fits in PRs and code reviews. A team can maintain this alongside the docs as they evolve.
- knowledge-base/notes/<topic>.md - one file per topic with full citations and cross-references. Use these for onboarding handoffs, audit prep, knowledge transfer.

Typical numbers (run on a substantial corpus, 80-150 documents, 4-6 subagents):

- Wall-clock time: 10-20 min depending on corpus size
- Topics extracted: 60-120 in a mid-size corpus
- Anchor concepts (cross-referenced 3+ times): 8-15
- Link resolve rate: aim for >95%

How to customize

Knob | Default | When to change

--- | --- | ---

Output folder | ./knowledge-base/ | Change to ./kb/ or ./docs-map/ if you collide with an existing folder

Partition count | 4-7 | Small focused corpus: 2-3. Multi-system or huge corpus: 8-10, but consider running twice with different roots instead

Default partitions | Architecture / Ops / Security / Onboarding / APIs / Domain / Process | Replace with domain-specific axes for your context - e.g., for a regulatory archive: "Regulation X / Audit Findings / Remediation Plans / Evidence"

Anchor count | 5-10 top-traffic topics | If you want a denser overview, raise to 15. If you want a tight executive summary, drop to 3

Source citation style | path:line-range (or path:page-range for PDFs) | If your docs are in a system with stable URLs (Confluence, Notion), substitute URL-based citations instead of path-based

Confidence labels | high / med / low | If your team uses a different vocabulary (verified / draft / hearsay), swap in your terms

Failure modes + recovery

Failure | Symptom | Recovery

--- | --- | ---

Corpus too large for one pass | Subagents return incomplete reports, context windows fill | Run twice with different roots: once on docs/architecture/, once on docs/operations/. Merge the two knowledge-base/ outputs by hand or with a follow-up synthesis pass.

Docs are mostly diagrams or screenshots | Subagents report "image-only, no extractable text" | Don't run this prompt - it's for text. Convert diagrams to text descriptions (Claude can OCR + describe) in a preprocessing pass first, then run this.

Lots of contradictions between docs | Open Questions list explodes (50+ entries) | Don't suppress them - that's a real finding about your docs. Triage in the synthesis pass: contradictions that block onboarding go first, stylistic inconsistencies go last.

Outdated source docs | Notes cite obsolete behavior | The prompt doesn't fix stale docs - it surfaces them. Add a "last-updated" check to the per-topic frontmatter and flag anything >12 months old as an Open Question.

Sensitive data in docs | Subagent reports it found credentials, PII, or unredacted financials | The prompt instructs subagents to flag-not-propagate. Verify the knowledge-base notes do NOT contain the sensitive content. Then escalate the source doc to the appropriate owner.

HTML mind-map looks flat | All topics on one level, no hierarchy | The corpus probably IS flat (no nested structure). That's an honest finding. Don't add fake hierarchy.

Per-topic notes get repetitive | Same boilerplate phrasing across 60 files | Add a one-line "writing style" directive to the prompt: "Each note should be readable as a standalone paragraph - avoid templated openings like 'This topic covers...'"

How this maps to the workshop

This is one of two reusable take-home prompts from the ITSS workshop:

- Repo Architecture Mapping (Repo_Architecture_Mapping_Prompt.md) - for codebases
- Documentation Mind-Map (this one) - for documentation corpora

Same methodology: parallel subagent dispatch, file:line discipline, validation pass, single command. Different units of work.

Apply it on Monday to:

- A legacy team's Confluence space when you're onboarding to it
- An exported regulatory archive (banking compliance, GDPR, internal audit)
- A /docs folder in a repo nobody on the team fully remembers
- A vendor's API documentation set before you commit to integration
- Your own team's internal docs as a knowledge audit - what's there, what's missing, what contradicts

The output is the artifact your team takes back. The methodology - parallel subagents, source-citation discipline, dual HTML + markdown outputs - transfers across any future tooling.

ITSS Workshop - 28 May 2026. Take-home prompt. Free to copy, modify, redistribute internally.